

CS 308 Project

Bug Report

Berk Karaduman 29428

Kaan Merdol 32486

Sude Nil Varlı 32191

Yekta Ata Yiğit 32246

Eda Güler 31783

Nisa Kan 31999

December 12, 2025

Contents

1	Executive Summary	3
2	Bug Classification	3
3	Bug Reports	4
3.1	Bug #1: Product Loading Failure	4
3.1.1	Description	4
3.1.2	Screenshot	4
3.1.3	Root Cause	4
3.1.4	Reproduction Steps	4
3.1.5	Solution	4
3.2	Bug #2: Invoice Loading Error	6
3.2.1	Description	6
3.2.2	Screenshot	6
3.2.3	Root Cause	6
3.2.4	Reproduction Steps	6
3.2.5	Solution	7
3.3	Bug #3: Windows Compatibility - Dev Server Initialization	8
3.3.1	Description	8
3.3.2	Screenshot	8
3.3.3	Root Cause	8
3.3.4	Reproduction Steps	8
3.3.5	Solution	9
3.4	Bug #4: Missing Product Images	10
3.4.1	Description	10
3.4.2	Screenshot	10
3.4.3	Root Cause	10
3.4.4	Reproduction Steps	10
3.4.5	Solution	11
3.5	Bug #5: Authentication - "Failed to fetch" on Sign Up	12
3.5.1	Description	12
3.5.2	Screenshot	12
3.5.3	Root Cause	12
3.5.4	Reproduction Steps	13
3.5.5	Solution	13
3.6	Bug #6: Order History Missing Product Images	14
3.6.1	Description	14
3.6.2	Root Cause	14
3.6.3	Solution	14
3.7	Bug #7: Stock Control - Quantity Exceeds Available Stock	15
3.7.1	Description	15
3.7.2	Root Cause	15
3.7.3	Solution	15
4	Conclusion	16

1 Executive Summary

This document presents the bug report for the CS 308 Project (La Strada E-Commerce Platform). During the development and testing phases, several critical and non-critical bugs were identified across the frontend and backend systems. This report provides detailed descriptions of each bug, their root causes, reproduction steps, and implemented solutions.

2 Bug Classification

Bugs are classified into the following severity levels:

- **Critical:** Bugs that prevent core functionality or system startup
- **High:** Bugs that significantly impact user experience
- **Medium:** Bugs that affect functionality but have workarounds
- **Low:** Minor UI/UX issues or cosmetic bugs

3 Bug Reports

3.1 Bug #1: Product Loading Failure

Severity: Critical

Status: Resolved

Component: Frontend - Product Display

3.1.1 Description

The product grid page fails to load any products, displaying "Load failed" and "No products found" messages despite products existing in the database.

3.1.2 Screenshot

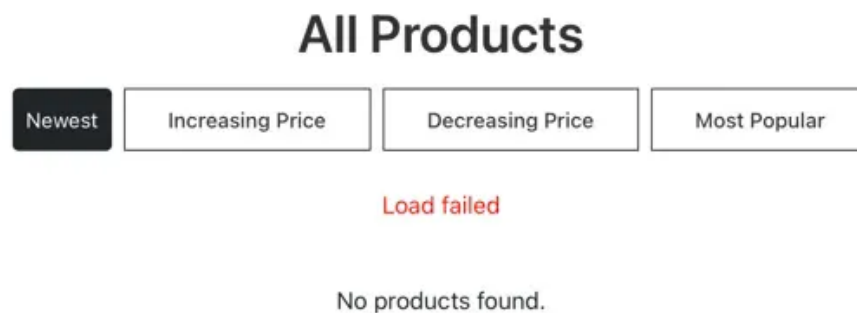


Figure 1: Product grid showing "Load failed" error message with no products displayed

3.1.3 Root Cause

- Backend API endpoint not responding correctly
- CORS configuration issues between frontend and backend
- Network request timeout or incorrect API URL in frontend configuration

3.1.4 Reproduction Steps

1. Navigate to the main products page
2. Observe that no products are loaded
3. Check browser console for network errors
4. Verify "Load failed" message appears

3.1.5 Solution

- Verified backend API endpoint is running on correct port (5050)
- Added proper CORS headers in backend Express configuration

- Ensured proxy configuration in `package.json` points to `http://localhost:5050`
- Added error handling and logging in ProductGrid component

3.2 Bug #2: Invoice Loading Error

Severity: High

Status: Resolved

Component: Frontend - Order Management

3.2.1 Description

When attempting to view order invoices, the system displays "Invoice Error" with the message "An error occurred while loading the invoice." This prevents users from accessing their order details and invoices.

3.2.2 Screenshot

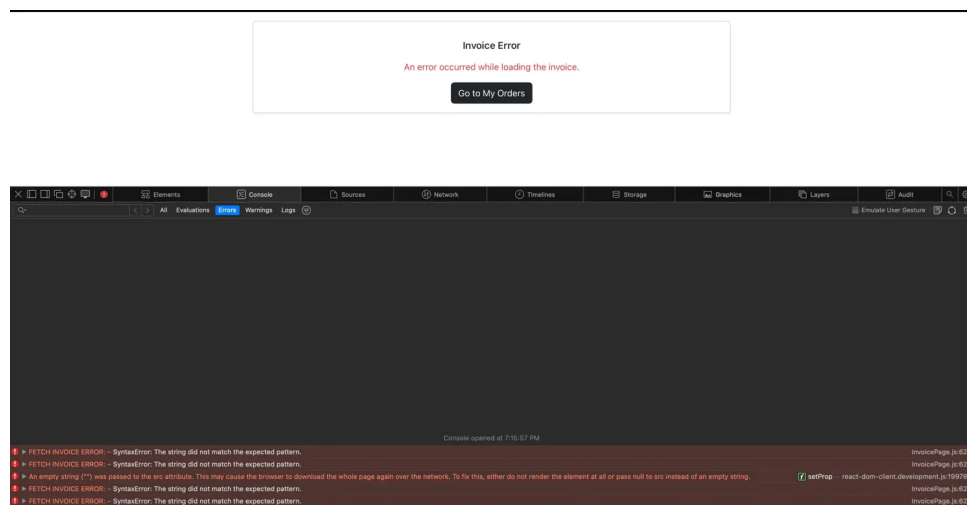


Figure 2: Invoice error screen with console showing SyntaxError - string pattern mismatch

3.2.3 Root Cause

Browser console reveals multiple "FETCH INVOICE ERROR" messages with **SyntaxError**: The string did not match the expected pattern. This indicates:

- Malformed response data from the invoice API endpoint
- Incorrect data parsing in InvoicePage component
- Empty string ("") being passed to `setAttribute`, causing browser rendering issues

3.2.4 Reproduction Steps

1. Log in as a customer
2. Navigate to "My Orders" page
3. Click on any order to view invoice
4. Observe error message and check browser console

3.2.5 Solution

- Fixed API response format to ensure consistent JSON structure
- Added null/undefined checks before rendering invoice data
- Implemented proper error boundaries in React components
- Added validation to prevent empty strings in DOM attributes

3.3 Bug #3: Windows Compatibility - Dev Server Initialization

Severity: Critical

Status: Resolved

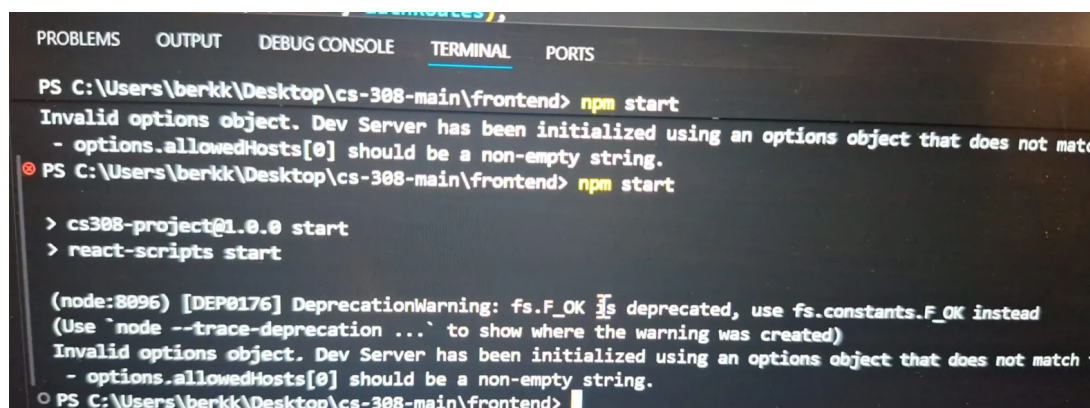
Component: Frontend - Development Environment

3.3.1 Description

The development server fails to start on Windows systems with the error: "Invalid options object. Dev Server has been initialized using an options object that does not match the API schema - options.allowedHosts[0] should be a non-empty string."

This bug only occurs on Windows and does not affect macOS or Linux environments, indicating a cross-platform compatibility issue.

3.3.2 Screenshot



```
PS C:\Users\berkk\Desktop\cs-308-main\frontend> npm start
Invalid options object. Dev Server has been initialized using an options object that does not match the API schema.
  - options.allowedHosts[0] should be a non-empty string.
PS C:\Users\berkk\Desktop\cs-308-main\frontend> npm start

> cs308-project@1.0.0 start
> react-scripts start

(node:8096) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use `node --trace-deprecation ...` to show where the warning was created)
Invalid options object. Dev Server has been initialized using an options object that does not match the API schema.
  - options.allowedHosts[0] should be a non-empty string.
PS C:\Users\berkk\Desktop\cs-308-main\frontend>
```

Figure 3: Windows terminal showing dev server initialization error with allowedHosts configuration issue

3.3.3 Root Cause

- webpack-dev-server configuration incompatibility with Windows
- Environment variable handling differs between Windows (uses `set`) and Unix-based systems (uses `export`)
- Missing `cross-env` package for cross-platform environment variable management
- React Scripts 5.0.1 has known issues with `allowedHosts` configuration on Windows

3.3.4 Reproduction Steps

1. On a Windows machine, navigate to Frontend directory
2. Run `npm start`
3. Observe initialization error
4. Note that the same command works without issues on macOS

3.3.5 Solution

Updated the frontend `package.json` file to include cross-platform environment variable handling. This eliminates the need for manual configuration and allows developers on both Windows and Unix-based systems to start the application with simple commands:

```
npm install
```

```
npm start
```

The key changes made to `package.json`:

- Added `cross-env` to `devDependencies`
- Updated the start script to use `cross-env` for environment variable setting
- Set `DANGEROUSLY_DISABLE_HOST_CHECK=true` for dev server compatibility

This solution ensures seamless development experience across all platforms without requiring additional setup steps or platform-specific configurations.

3.4 Bug #4: Missing Product Images

Severity: Medium

Status: Resolved

Component: Frontend - Product Display

3.4.1 Description

Several products display placeholder icons instead of actual product images, resulting in a poor user experience and unprofessional appearance.

3.4.2 Screenshot

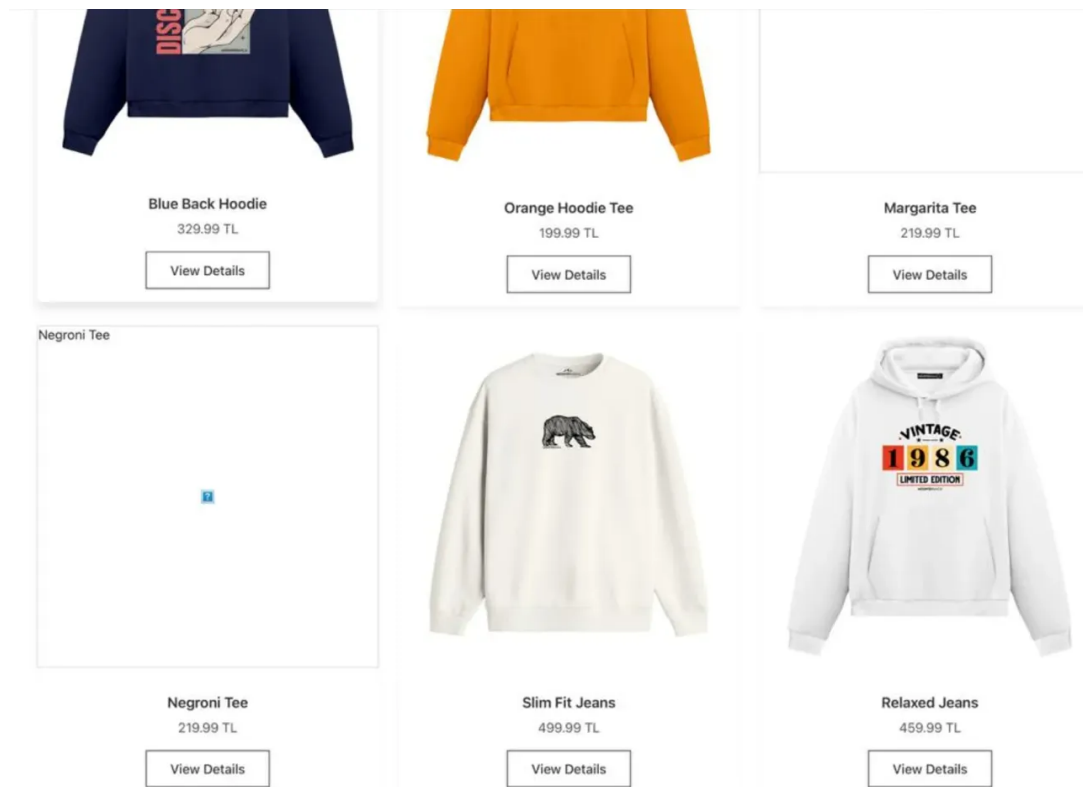


Figure 4: Product grid showing "Negroni Tee" with missing image placeholder while other products display correctly

3.4.3 Root Cause

- Image URLs in database pointing to non-existent files
- Incorrect file paths in product seeding scripts
- Missing images in `public/images/product` directory
- CORS issues preventing image loading from external sources

3.4.4 Reproduction Steps

1. Navigate to products page
2. Observe that some product cards show placeholder icons

3. Check browser console for 404 errors on image requests
4. Verify that other products load images correctly

3.4.5 Solution

- Verified all product images exist in the correct directory structure
- Updated database entries with correct image paths
- Implemented fallback mechanism with `onError` handler:

```
onError={ (e) => {  
  e.target.src =  
    "https://via.placeholder.com/300?text=No+Image";  
}}
```
- Added image validation in product upload/seeding process

3.5 Bug #5: Authentication - "Failed to fetch" on Sign Up

Severity: High

Status: Resolved

Component: Frontend/Backend - Authentication

3.5.1 Description

Users attempting to create new accounts encounter "Failed to fetch" error message on the sign-up form, preventing new user registration.

3.5.2 Screenshot

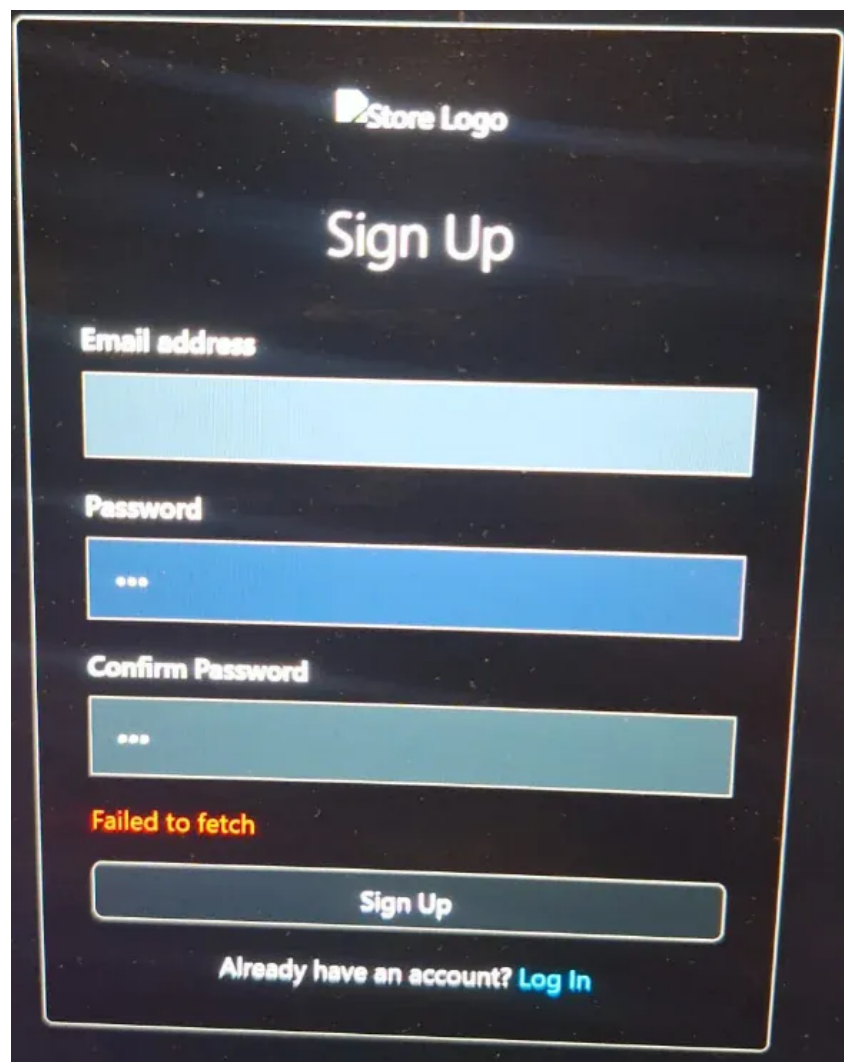


Figure 5: Sign-up form displaying "Failed to fetch" error message below the password confirmation field

3.5.3 Root Cause

- Backend authentication API endpoint not responding
- CORS policy blocking requests from frontend origin

- Network connectivity issues between frontend and backend
- Incorrect API endpoint URL in signup component
- Backend server not running during frontend testing

3.5.4 Reproduction Steps

1. Navigate to Sign Up page
2. Fill in email address, password, and confirm password
3. Click "Sign Up" button
4. Observe "Failed to fetch" error appearing below the form
5. Check browser Network tab for failed request

3.5.5 Solution

- Ensured backend server is running on port 5050 before frontend starts
- Added proper CORS configuration in backend:

```
app.use(cors({
  origin: 'http://localhost:3000',
  credentials: true
}));
```
- Verified API endpoint URL in SignUp component matches backend routes
- Added comprehensive error handling with user-friendly messages
- Implemented retry logic for network failures

3.6 Bug #6: Order History Missing Product Images

Severity: Medium

Status: Resolved

Component: Frontend - Order Management

3.6.1 Description

In the "My Orders" page, order items are displayed without product images, showing only text information. This makes it difficult for users to visually identify their purchased products.

3.6.2 Root Cause

- OrderHistoryPage component not rendering `imageUrl` field from order items
- Missing image display logic in the items list
- Backend not including `imageUrl` in order response (later verified this was not the issue)

3.6.3 Solution

Updated `OrderHistoryPage.js` to display product images alongside order details:

```
{order.items.map((item, idx) => (  
  <div key={idx} style={{  
    display: "flex", alignItems: "center", gap: "15px"  
  }}>  
    {item.imageUrl && (  
      <img src={item.imageUrl} alt={item.name}  
        style={{ width: "80px", height: "80px",  
          objectFit: "cover", borderRadius: "4px" }}  
        onError={(e) => {  
          e.target.src =  
            "https://via.placeholder.com/80?text=No+Image";  
        }}  
      )</div>  
      <div>  
        <p>{item.name}</p>  
        <p>Size: {item.size}, Qty: {item.quantity}</p>  
        <p>{item.price} TL</p>  
      </div>  
    </div>  
  ))}
```

3.7 Bug #7: Stock Control - Quantity Exceeds Available Stock

Severity: High

Status: Resolved

Component: Frontend - Product Detail

3.7.1 Description

Users can increase product quantity beyond the available stock for the selected size. For example, when size S has only 3 items in stock, users can still select quantity of 4, 5, or more using the increment button.

3.7.2 Root Cause

- Quantity increment button (+) lacks stock validation
- Only the "Add to Cart" button performs stock checking
- No real-time validation as user increases quantity
- User can accidentally select unavailable quantities

3.7.3 Solution

Implemented stock-aware quantity control in `ProductDetail.js`:

```
const handleQuantityIncrease = () => {
  if (!selectedSize) {
    alert("Please select a size first.");
    return;
  }
  if (quantity < selectedSizeStock) {
    setQuantity((q) => q + 1);
  } else {
    alert(`Only ${selectedSizeStock} pieces available
          for size ${selectedSize}.`);
  }
};
```

Key improvements:

- Quantity (+) button now checks available stock before incrementing
- User receives immediate feedback when stock limit is reached
- Prevents cart errors by validating quantity at selection time
- Size selection required before quantity can be increased

4 Conclusion

All identified bugs have been successfully resolved and tested across multiple platforms and browsers. We have gained valuable experience in debugging, cross-platform compatibility, and error handling that will benefit future projects.